

Distributed File System for Big Data

 systemdesignschool.io/fundamentals/distributed-file-systems



Distributed File Systems

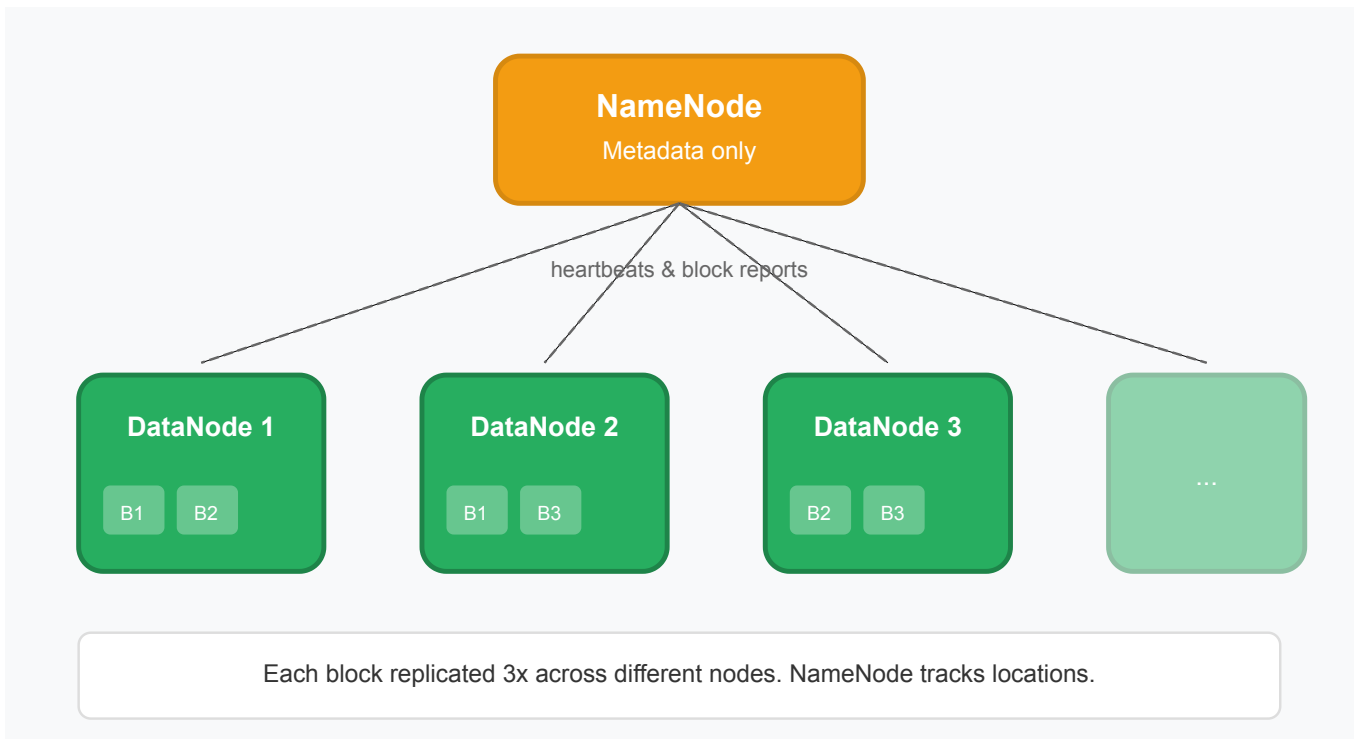
The previous article showed how MapReduce distributes computation across a cluster. But where does the data live? A single machine can't hold petabytes. MapReduce needs a storage layer that spans thousands of machines while handling hardware failures gracefully.

HDFS (Hadoop Distributed File System) solves this problem. It stores massive files across commodity hardware—regular servers with standard disks—while providing fault tolerance through replication.

The Core Design: Blocks and Replicas

HDFS breaks large files into fixed-size **blocks** (typically 128MB each). Each block is replicated across multiple machines (3 copies by default). If a disk fails, the data remains available from other replicas.

Why 128MB blocks? Two reasons. First, it reduces metadata overhead—a petabyte file has only ~8 million blocks to track, not trillions of small file entries. Second, it amortizes disk seek time. Reading 128MB sequentially is efficient; reading thousands of tiny files means thousands of seeks.



Two Types of Nodes

The cluster has two roles, separated for scalability.

DataNodes store actual data blocks. Each commodity server runs a DataNode daemon that manages local disk storage and serves block reads/writes over the network. A production cluster might have thousands of DataNodes, providing aggregate storage in the petabyte range.

NameNode tracks metadata: which files exist, which blocks comprise each file, and which DataNodes currently hold each block's replicas. The NameNode keeps all metadata in memory for fast lookups—a critical design choice that enables handling millions of file operations per second.

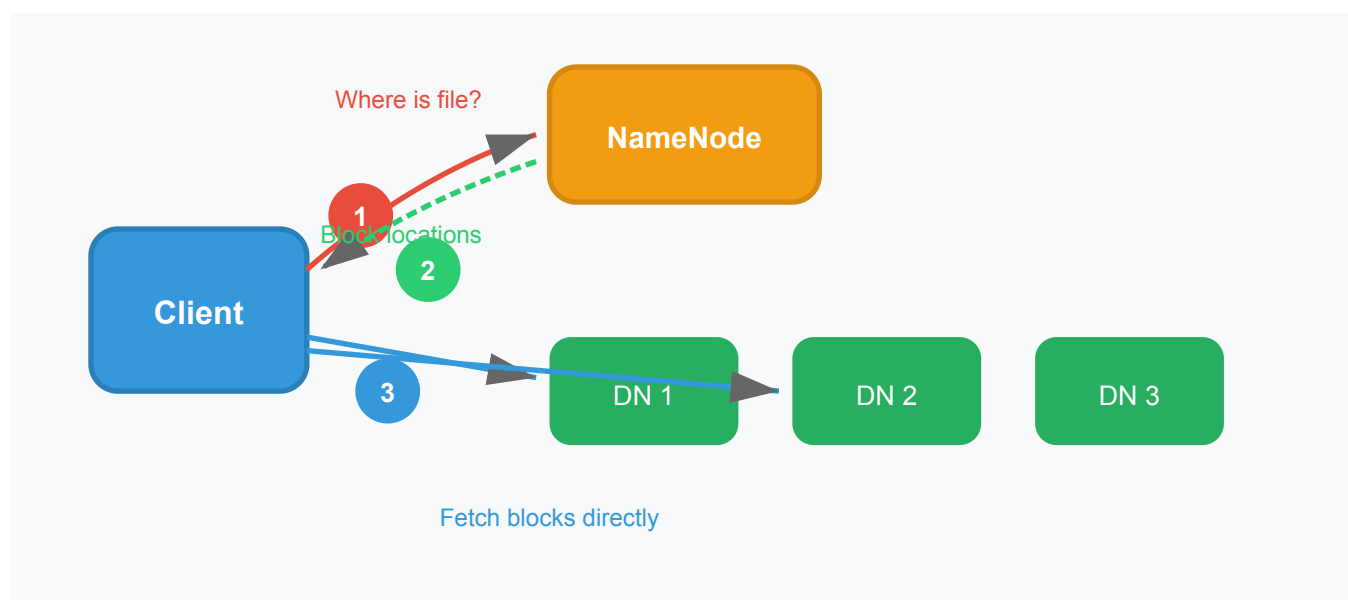
Why Separate Metadata from Data?

This separation is the key to HDFS scalability. The NameNode handles lightweight lookups (where are the blocks?) while DataNodes handle bulk data transfer (read/write the actual bytes).

When a client reads a file, it asks the NameNode for block locations, then fetches blocks directly from DataNodes. The NameNode never touches actual file contents—it only returns addresses. This means you can add more DataNodes for more storage and throughput without overloading the NameNode.

Reading a File

Let's trace what happens when a MapReduce job reads an input file.



1. **Client contacts NameNode:** "Where are the blocks for `/data/logs/access.log`?"
2. **NameNode returns metadata:** Block IDs, sizes, and which DataNodes hold each replica
3. **Client fetches blocks directly:** For each block, the client connects to a DataNode (preferring the closest replica) and streams the data
4. **Client assembles the file:** Blocks are read in order and concatenated

The NameNode is never on the data path. It handles one lightweight RPC at the start, then steps aside while terabytes flow between client and DataNodes.

Writing a File

Writing uses a clever **pipeline replication** scheme to minimize client network overhead.

1. **Client requests file creation:** NameNode allocates block IDs and selects DataNodes for replicas
2. **Client streams to first DataNode:** The client sends data to just one DataNode
3. **Pipeline to replicas:** The first DataNode forwards data to the second, which forwards to the third—forming a pipeline
4. **Acknowledgments flow back:** Each DataNode acknowledges after writing to local disk, and acks propagate back through the pipeline
5. **Client notifies NameNode:** After all blocks complete, the client tells the NameNode the file is finished

The client sends each byte only once. The DataNodes handle copying to replicas in the background, using the cluster's internal network bandwidth rather than the client's connection.

Fault Tolerance and Recovery

DataNode failures are routine—disks fail, machines crash, network partitions happen. Each DataNode sends periodic heartbeats to the NameNode. When heartbeats stop, the NameNode marks that DataNode as dead and triggers re-replication: it instructs other DataNodes to copy the affected blocks to healthy nodes, restoring the replication factor.

NameNode failure is more serious—it's a single point of failure. Production deployments run a **standby NameNode** that maintains a copy of the metadata. The primary and standby share an edit log (often stored on a quorum of JournalNodes). If the primary fails, the standby takes over, typically within seconds to minutes depending on configuration.

Rack awareness improves fault tolerance further. HDFS places replicas across different racks, so a rack failure (switch failure, power loss) doesn't lose all copies of a block.

Why This Matters for MapReduce

HDFS enables MapReduce's key optimization: **data locality**. The NameNode tracks which DataNodes hold which blocks. When the MapReduce scheduler assigns map tasks, it prefers nodes that already have the input data locally. Instead of moving terabytes across the network, the computation moves to the data.

This locality is why HDFS uses large blocks. A 128MB block contains enough data to keep a mapper busy for a meaningful time. Tiny blocks would mean constant task scheduling overhead without enough work per task.

So far we've covered the traditional Hadoop stack: MapReduce for computation, HDFS for storage. But MapReduce has limitations—particularly for iterative algorithms and multi-stage pipelines that repeatedly read and write intermediate results. That leads into Spark, which addresses these limitations by caching data in memory across stages.